X04｜换掉运行实例以后，怎样确认还能服务和恢复？

本章把运行关系接回已经出现的业务事实：**哪份程序 + 哪份配置 + 哪个持久对象 → 当前可观察行为。** 恢复本地数据库还要尊重合作方独立保存的历史。

实例换掉以后，究竟接上了哪份数据？

当前问题	关系或短用法	记到什么程度
image 相同，就会用同一份数据吗？	image 由源码、Dockerfile、依赖构建；Compose 另外给环境、端口与挂载，再创建实例。	模板不自动包含本次数据与挂载，配置也不把现有数据变成期望内容。
应用与合作方各写到哪里？	API：`/data/tickets.db` → ticket-data；模拟合作方：`/partner-data/receipts.db` → partner-data。	**两个程序、两个持久所有者。** 应用 backup 只处理自己的库。
`/data`、tickets.db 与 named volume 是一个对象吗？	分别是容器挂载位置、应用文件、Docker 管理的存储对象。	认清文件归属；`/data` 不是固定 Windows 目录。
从 Windows 与从 API 调合作方，走同一路吗？	Windows 经主机 18080 转 API 8000；API 经 `fake-partner:9000` 找共同网络的服务。	调用位置决定地址；API 的 localhost 只指自己。详细端口关系回 F03。
重建后怎么证明用了原数据？	核对新实例身份与实际 mount，GET 同一工单并检查内容。	只看名称或新建成功，不足以证明旧事实保留。
volume 被删除后，再建同名就恢复了吗？	同名字符串可以属于新的空对象生命周期；V1/V2/V3 要按实际基线登记。	**名称相同不等于对象或数据相同。**

回看：「从源码到文件路径，中间有哪些真实对象？」「普通、降级和版本回归为什么需要不同基线？」。

知道版本、ready 与基线以后，可以宣布哪些结果？

当前问题	关系或短用法	记到什么程度
D1、D2 只需记录 tag 吗？	保存完整 local image ID，再查运行 container 的 `.Image`；真实 registry 分发且确有 RepoDigest 才登记。	精确身份与可移动标签分开，base digest 不冒充应用身份。
C1/C2 如何识别运行差别？	对按指定顺序合并的 Compose 展开 JSON 文件字节取指纹，同时保留文件与工具来源。	hash 帮助发现变化，不替代配置文件，也不公开秘密。
X 的 ready 覆盖了 F 的全部检查吗？	X 查必要数据读取、tickets 可访问和 schema 版本；未逐项核全部列/约束/索引，也未证明所有写能力。	**不同应用的同名 ready 不可套用范围。** F 的结构与短写探针不自动搬来。
合作方不可用，为什么 ready 仍可通过？	X 最低承诺允许本地保存并转人工，合作方是可降级依赖。	先依据合同决定能力；实际合同要求确认后才接单时需另设计。
正常 UAT 的空基线只有应用库吗？	应用库与合作方 receipt store 都要符合空基线，才能解释 T-42、PA-1 和一次派单。	把两边历史都算进前提，不只重置本地。
想验降级，用已成功 key 重跑行吗？	已收束意图直接读回，不会调用坏合作方；独立降级用另一个 project、端口、两份 volume 与独立 key。	选择能真正触发目标分支的基线；改 request ID 不能把旧意图变成新场景。

running、live、ready、创建/重放/读回/政策 UAT、备份恢复分别回答不同问题；等待健康也应有明确上限。完整身份、等待与场景配置回看「哪份程序配哪份数据，不能只凭版本名猜」「能响应、准备接单、业务正确是不同的观察」「普通、降级和版本回归为什么需要不同基线？」。

B1 恢复成可用数据，哪些证据不能合并？

当前问题	关系或短用法	记到什么程度
B1 文件存在且 hash 匹配，就证明恢复成功吗？	先用 SQLite backup API 取得一致副本，登记来源；再恢复到原本不存在的新目标，核完整性与 schema，最后由隔离 API 读回。	副本身份、数据库可用、业务读回是连续但不同的证据。
恢复文件 hash 必须等于备份文件吗？	数据库恢复后物理布局可不同；本例核备份源 hash、恢复 integrity_check 与登记业务字段。	**数据库恢复不以字节相等为定义。**
默认 restore readback 具体证明什么？	GET T-42，核设备、主题、priority=3、ticket_status=open；没有新建，也没有重验外部降级或 image 回滚。	写明读到的字段，不扩成全部业务规则与完整恢复验收。
默认隔离脚本会改原数据吗？	`Invoke-IsolatedRestoreDrill.ps1` 核来源 image/config，建带 label 的新 volume/隔离 API，拒绝覆盖已有目标，默认不联系合作方。	明确认读目标与资源副作用；成功只清自己的匹配资源，失败保留现场。
schema=1 与 integrity_check=ok 意味着什么？	说明相应版本元数据与数据库完整性检查通过；不表示全部 DDL、约束和业务规则已核对。	不把元数据标签或数据库检查扩成业务证明。

完整备份与默认隔离读回路线回看「一份 B1，怎样从文件变成可恢复证据？」。命令在明确实验副本中按 RUNBOOK 查用；restore 的目标必须是新对象，不能把旧数据库当练习目的地。

应用恢复了，合作方的历史为什么还在？

当前问题	关系或短用法	记到什么程度
从 B1 恢复 tickets.db，会撤销 PA-1 吗？	PA-1 属于合作方 receipts.db；应用备份没有它，也没有权力让外部世界一起回到备份时点。	**本地恢复与远端效果直接分开。**
合作方重启后，如何仍认得旧意图？	本例把 key、payload 摘要、派单编号与计数持久保存；原 key/内容返回 PA-1，新意图得到下一项。	幂等前提是持久 receipt，不只是内存计数或看起来独特的随机编号。
合作方库丢了，恢复应用 B1 足够吗？	已登记恢复路线依赖原 store 继续存在；前提失效需核对外部事实及它的恢复机制。	不能宣称本地备份修好了独立所有者。
恢复后为什么先 replay 旧票再创建新票？	旧意图应回 T-42、无新派单；新 key 在恢复库得到 T-43，并由原合作方 store 产生 PA-2。	核对两份独立历史继续协调；没有共同事务或共同回滚。
`-RunRestoredUat` 比默认 readback 多做什么？	接入已核对的合成合作方网络，先验旧 replay，再验新 T-43 创建/重放/读回/政策。	**这是新写入，会产生一项远端效果。** 清理隔离 API 不撤销 PA-2。
恢复后普通 UAT 通过，可覆盖所有失败路线吗？	当前子集不含 always_500、close_proposal 权限拒绝或版本回滚；需要时分别给基线与预期再验。	readback、恢复后指定 UAT、rollback 分别报告。

回看：「恢复应用，为什么不能让外部世界一起回到过去？」「恢复后写入，要另做一次什么验证？」。主机双进程验证和容器手册是不同环境的证据，不能互相冒充已运行。

ready 通过的 D2 为什么仍应退回，交接怎样说清？

当前问题	关系或短用法	记到什么程度
X 的受控 D2 哪里坏了？	uat-regression 仍可读 schema，live/ready 可 200；合法创建在意图事务前返回 503/training_create_regression。	**健康通过不能代替客户路线。** 与 F 的 D2 schema_mismatch 场景分清。
只见 503 就能断定这个回归吗？	还需具体 detail、无新增票/事件/派单、编号未消耗，才符合本次受控场景。	由触发条件与状态证据共同定位。
退回 D1 为什么不顺手 restore B1？	本例 D2 没有破坏性迁移；确认兼容后换回 D1/C1，核实际 Image、ready 与整条新 UAT。	**程序配置回滚不自动需要数据恢复。** 已迁移数据必须另判兼容与新增事实。
D2/D1 回归的 T-42 是找回旧 V1 吗？	本路线在明确的新 V3 基线上进行；沿失败意图原 key/payload 创建 T-42，说明 D2 未消耗编号。	不把新基线成功写成已找回被清理的旧数据。
并发 finalization 后，回复依据谁？	以实际已提交状态为准；后完成者记录 assignment_finalize_replayed，不虚构第二次 commit。	恢复或回滚不跳过原有状态保护。
runbook 要交出哪些判断？	实验目录、project/端口、image/config/data 身份、输入与期望、失败停止处、保留及清理精确对象的方式。	让下一操作者无需猜前提；完整命令按阶段查，不机械串接。
`down --volumes` 会影响谁？	本 project 相关 volume，包括应用库和模拟合作方库；普通 down 默认保留 named volume。	数据丢弃是单独决定，需精确 label、内容与恢复安排；不能用 prune 处理其他项目。

回看：「D2 可以 ready，为什么仍然不能采用？」「让另一位操作者可以按说明做完」。最终记录注明目标环境、对象身份、实际输出和结论范围；未运行仍是未验证，失败仍按失败登记。静态声明检查、Fake 计数、主机进程测试均不能写成真实容器网络与挂载已通过。

跨册查找时，先说清现在不确定的交接：文字变值回 A；HTTP 消息与回应检查回 B；行/事件/计数共同提交回 C；未知外部效果与稳定意图回 D；资料、候选和动作授权回 E；image/config/volume 与恢复回 F。它们在同一应用的多个位置起作用，不是一条 A→B→C→D→E→F 的请求流水线。整体关系见原教材「关系收束 | 从一个现象找到下一项有用证据」。